

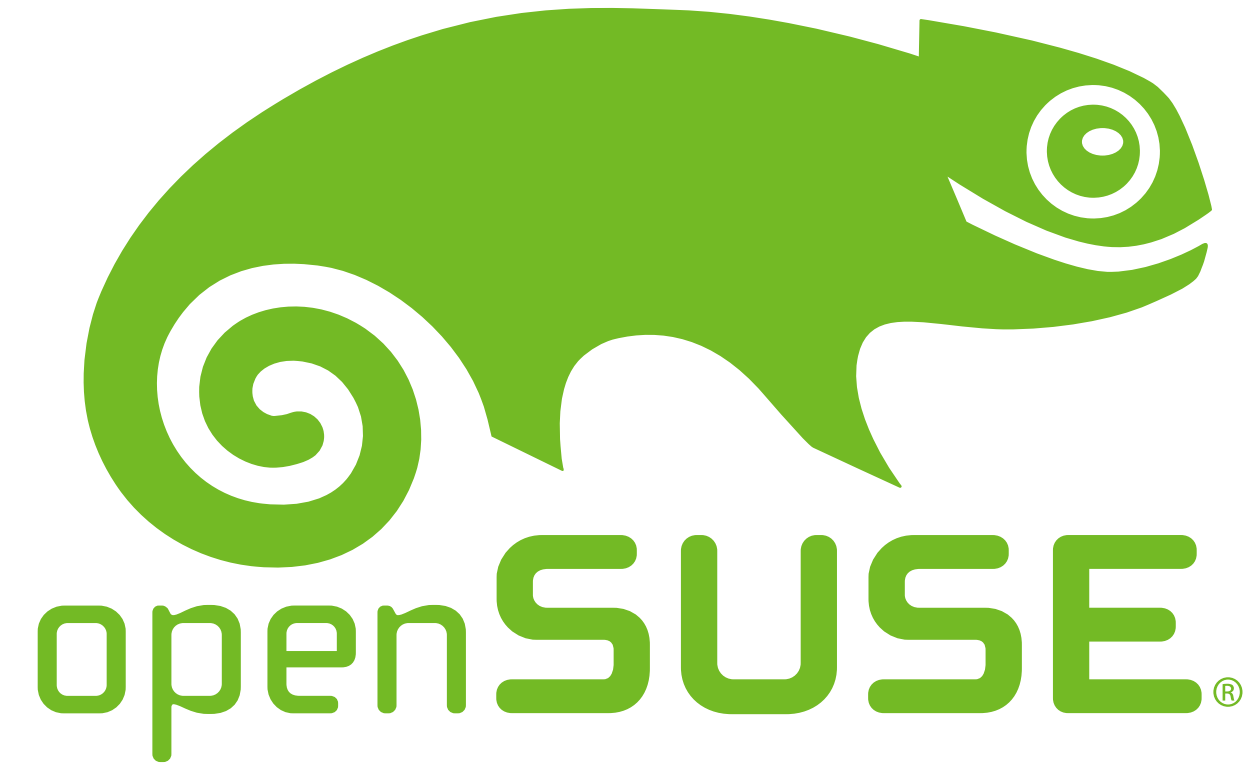
Andrea Manzini

Green Computing

Misurare il Consumo Energetico

su Linux  





Mi presento

Andrea Manzini

<https://ilmanzo.github.io>

- Veteran Unix Admin ; BOFH
- Software engineer @SUSE
- Open Source Contributor 🙋
- Package Maintainer 📦

Cosa vedremo oggi

- Perché è importante?
- Concetti Fondamentali: La differenza tra Potenza ed Energia.
- Come funziona? Uno sguardo alla tecnologia hardware/software
- Gli Strumenti del Mestiere
- Demo Live: Misuriamo un programma reale.
- Limiti e Conclusioni.

Perché l'efficienza energetica è importante ?

Non è solo una questione di ecologia/moda "green"

➡️ Dispositivi Mobili & IoT

🖧 Data Center & Cloud

🚀 Prestazioni & Calore



🤔 Watt, Joule?

Potenza [Watt]:

È la velocità con cui l'energia viene consumata. Es. la velocità della tua auto (sto andando a 90 km/h).

Energia [Joule]:

È la quantità totale di potenza usata in un intervallo di tempo. Es. la distanza totale percorsa (ho percorso 200 km).

$$\text{Energia} = \text{Potenza} \times \text{Tempo}$$

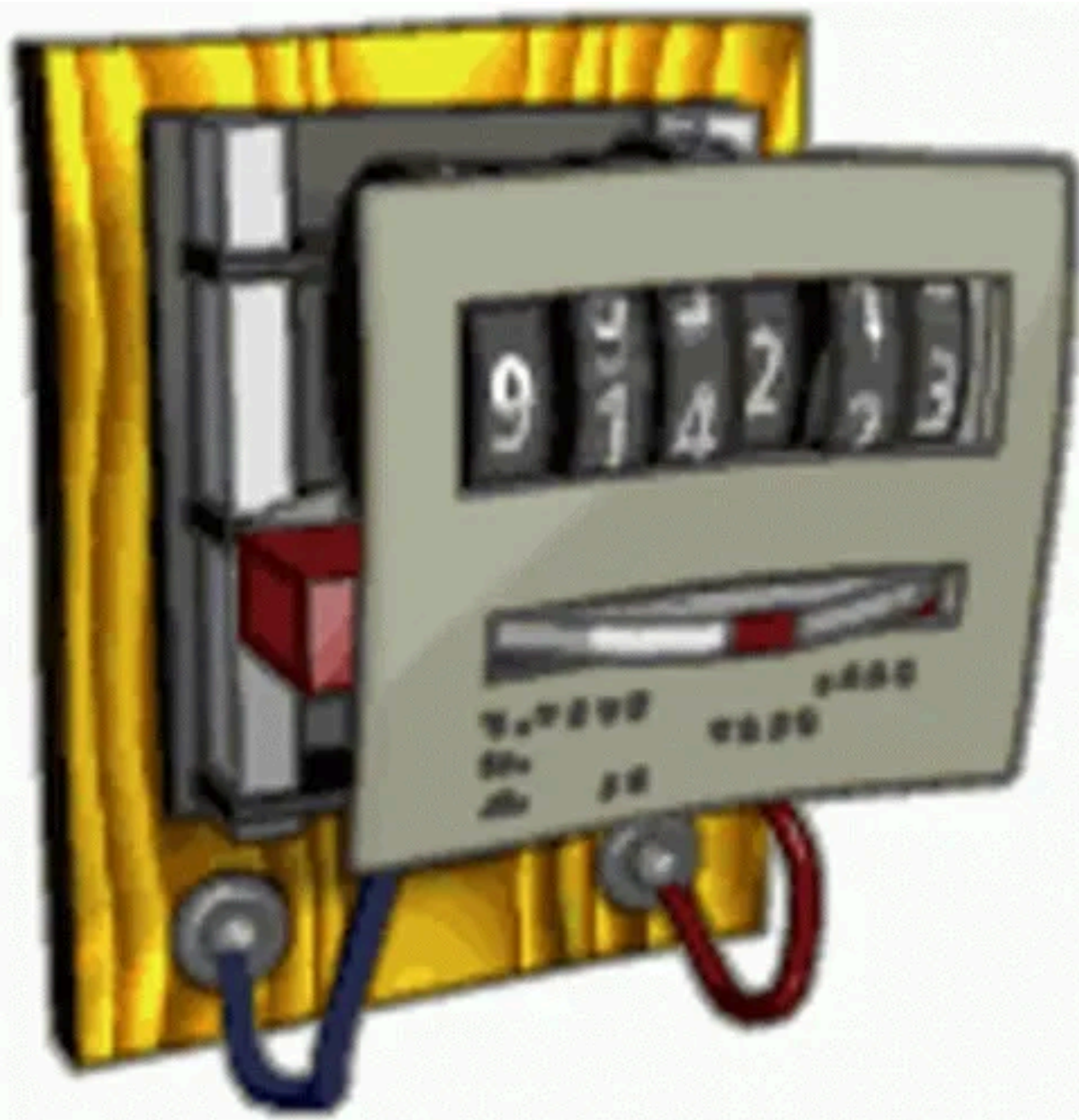
Si, ma in pratica ?

ripasso di fisica: $1W = 1 \text{ J/s}$ (energia consumata al secondo)

$1\text{kWh} = 1000 \text{ Watt per } 1 \text{ ora} = 1000 * 3600 * \text{J} = 3.6\text{e}6 * \text{J}$

un kWh in Italia costa circa 0,16 centesimi tasse incluse (*molto* a spanne)

- caricare un dispositivo a 30W per 6 ore tutti i giorni consuma $365 * 0,18 \text{ kWh} = 65\text{kWh}$ cioe' 10 euro l'anno
- un pc acceso + monitor consuma in media circa 100 W, se lo usiamo tutti i giorni 10 ore al giorno spendiamo circa 60 euro l'anno
- Un Datacenter consuma anche 100 MW, Nei prossimi anni si prevede che i data center raggiungeranno un consumo energetico pari a 1 GW entro il 2026



La Magia sotto il cofano

Come può il software misurare l'hardware?

Il software da solo non può misurare l'assorbimento elettrico. Si affida a dei contatori e a modelli energetici esposti direttamente dal processore.

Intel RAPL (Running Average Power Limit)

Cos'è? Un'interfaccia presente nelle CPU moderne (Intel e AMD) che fornisce stime sul consumo energetico.

È accurato? Non è una misurazione fisica, ma un modello software molto preciso, basato su contatori di attività hardware, voltaggi e temperature. Per i nostri scopi, è più che sufficiente.

Cosa misura? Espone diversi "domini" di consumo:

- PKG: L'intero socket della CPU.
- CORE: I core computazionali.
- UNCORE: La cache L3, i memory controller, ecc.
- DRAM: I moduli di RAM collegati.

RAPL in Linux: il framework powercap

Linux integra i dati di RAPL attraverso il framework powercap.

I dati sono accessibili direttamente tramite il filesystem virtuale, in:

```
/sys/class/powercap/
```

Dentro questa directory, troverete delle sottodirectory per ogni interfaccia (es. `intel-rapl:0`) con file come `energy_uj` (energia in **microJoule**) che possono essere letti.

Di seguito vediamo alcuni strumenti che semplificano la lettura e l'interpretazione di questi file.

Un primo script molto grezzo

```
import time

FILE = "/sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj"

def read_energy():
    with open(FILE, "r") as f:
        return int(f.read().strip())

v0 = read_energy()
while True:
    time.sleep(1)
    v1 = read_energy()
    print(f"Potenza media: {(v1-v0)/1e6:.1f} W")
    v0 = v1
```

powertop, powerstat: i "cruscotti" del sistema

Esistono dei tool per monitorare il consumo energetico in tempo reale.

Ideali per:

- Identificare processi e driver che "svegliano" la CPU inutilmente.
- Avere una visione d'insieme del consumo del sistema.
- Applicare al volo ottimizzazioni a livello di sistema (scheda "Tunables").

Comando: `sudo [powertop, powerstat]`

perf: Il Bisturi di Precisione

`perf` è lo strumento standard de facto per la profilazione delle performance (e dell'energia) su Linux.

Ideale per:

Ottenere una misura precisa e ripetibile in Joule per l'esecuzione di un singolo comando.

Confrontare il costo energetico di due versioni di un algoritmo.

Integrare la misurazione energetica in script di test o CI/CD.

Perf in azione

Usare perf per misurare l'energia

Il comando `perf stat` esegue un programma e, al termine, riporta le statistiche raccolte, incluse quelle energetiche.

```
sudo perf stat -e [eventi] -- [comando]
```

Esempio pratico:

```
# Misura l'energia consumata da CPU (pkg) e RAM (dram)
# durante l'esecuzione di 'mio_programma'

sudo perf stat -e power/energy-pkg/,power/energy-dram/ -- ./mio_programma
```

Dall'idea alla misura in 4 passi

- Prepara l'ambiente: Chiudi tutte le applicazioni non necessarie (browser, chat, etc.). Un ambiente "pulito" garantisce misure più stabili e ripetibili.
- Stabilisci una baseline: Misura il programma nella sua versione attuale per avere un punto di riferimento.
- Ottimizza: Applica le tue modifiche al codice, cambia gli flag di compilazione o l'algoritmo.
- Rimisura e Confronta: Esegui di nuovo la misurazione nelle stesse identiche condizioni e confronta i risultati in Joule.

Demo: misuriamo un ordinamento

Creo un file di testo da 1GB

```
$ tr -dc 'A-Za-z0-9 ' < /dev/urandom | fold -w 80 | head -c 1G > big.txt  
$ wc -l big.txt  
13256071 big.txt
```

```
$ time sort --parallel=1 big.txt > /dev/null  
24.244 total
```

```
$ time sort --parallel=16 big.txt > /dev/null  
6.597 total
```

La versione parallela finisce prima, ma usa tutti i processori. Consumerà meno energia totale?

Misura 1: sort single-thread

Comando:

```
sudo perf stat -e power/energy-pkg/ -- sort --parallel=1 big.txt > /dev/null
```

Performance counter stats for 'system wide':

```
455.22 Joules power/energy-pkg/  
25.19 seconds time elapsed
```


Misura 2: sort parallelo

Comando:

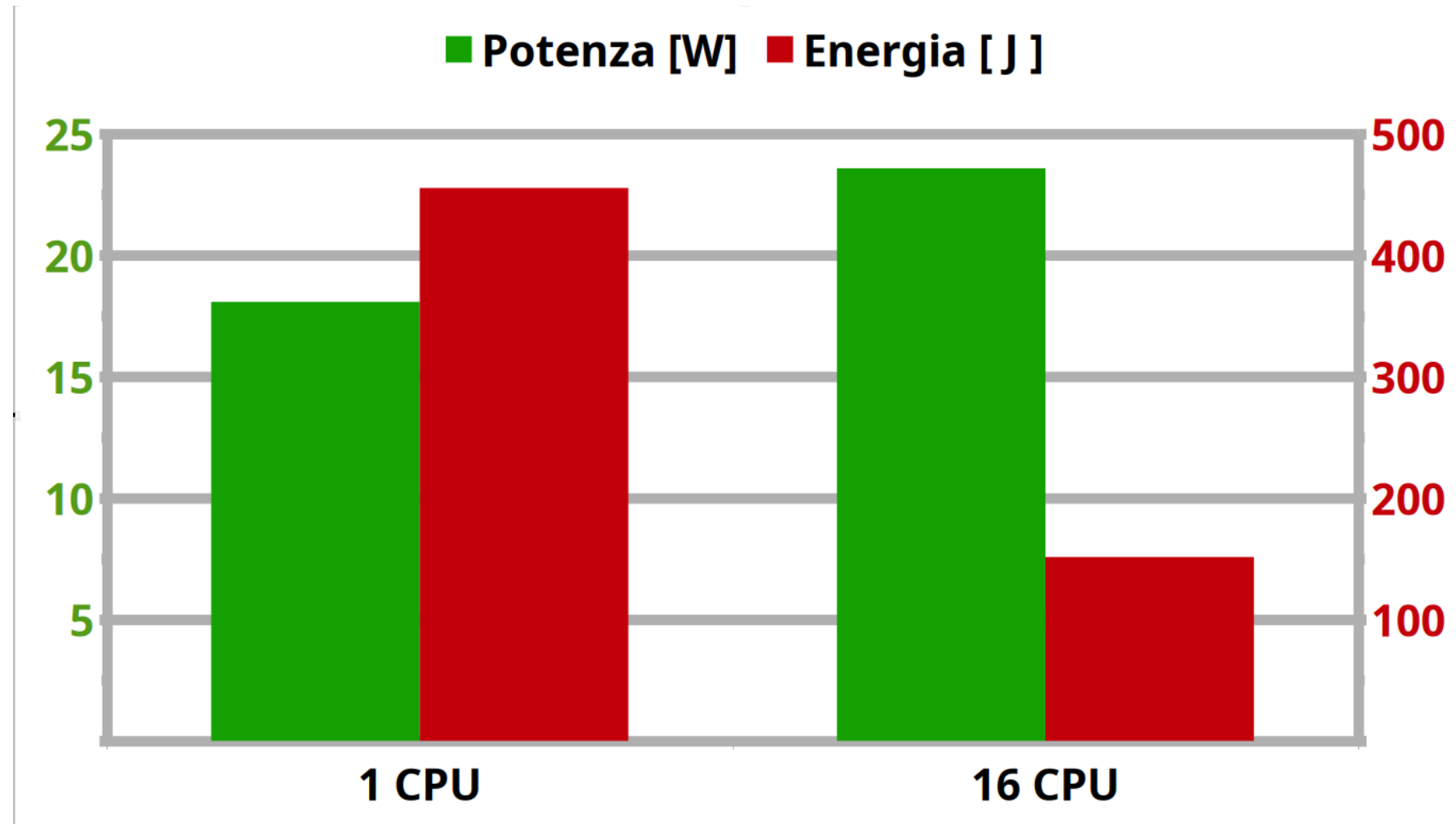
```
sudo perf stat -e power/energy-pkg/ -- sort --parallel=16 big.txt > /dev/null
```

Performance counter stats for 'system wide':

```
151.34 Joules power/energy-pkg/  
6.42 seconds time elapsed
```

Risultato

un risparmio
energetico del
~66%!



Il 1.000.000 numero primo

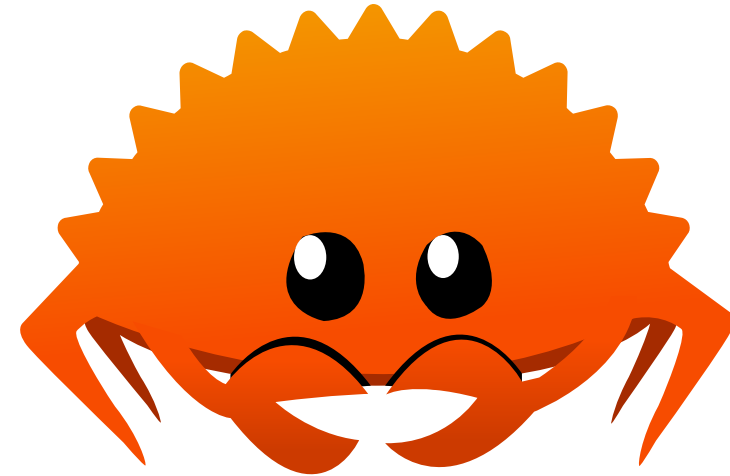


```
#!/usr/bin/env python3
from math import sqrt

def isPrime(n):
    for j in range(3, int(sqrt(n)+1), 2):
        if n % j == 0:
            return False
    return True

n, i = 2, 3
while n < 1_000_000:
    i += 2
    if isPrime(i):
        n += 1
print("Mth Prime is ", i)
```

```
fn is_prime(n: u64) -> bool {  
    let sqrt_n = (n as f64).sqrt() as u64;  
    for j in (3..=sqrt_n).step_by(2) {  
        if n % j == 0 {  
            return false;  
        }  
    }  
    true  
}  
  
fn main() {  
    let mut n = 2;  
    let mut i = 3;  
  
    while n < 1_000_000 {  
        i += 2;  
        if is_prime(i) {  
            n += 1;  
        }  
    }  
    println!("Mth Prime is {}", i);  
}
```



Pronti ? Via!

```
$ sudo perf stat -e power/energy-pkg/ -- ./prime.py
```

```
Mth Prime is 15485863  
Performance counter stats for 'system wide':  
934.24 Joules power/energy-pkg/  
50.23 seconds time elapsed
```

```
$ sudo perf stat -e power/energy-pkg/ -- prime_rs/target/release/prime
```

```
Mth Prime is 15485863  
Performance counter stats for 'system wide':  
39.28 Joules power/energy-pkg/  
2.15 seconds time elapsed
```

Per questo task, 🦀 é 23 volte piú veloce e consuma 1/23 di 🐍

Cosa NON stiamo misurando

È importante essere consapevoli dei limiti di questo approccio.

RAPL è un modello, non la verità assoluta. L'accuratezza può variare leggermente tra le architetture.

Copertura parziale: RAPL misura CPU e RAM, ma non copre:

- GPU dedicata
- Dischi (SSD/HDD)
- Scheda di rete

Per un'analisi completa dell'intero sistema, sarebbero necessari strumenti esterni (wattmetri fisici).

Cosa portarsi a casa

- L'efficienza energetica è una metrica di qualità del software tanto quanto le performance o la correttezza.
- La tecnologia chiave è RAPL, un'interfaccia hardware esposta da Linux.
- Usa `power top` per una diagnosi generale del sistema e per scovare "sprechi" in background.
- Usa `powerstat` per sapere quanto sta consumando il tuo sistema mentre funziona
- Usa `perf stat` per misurazioni precise e ripetibili del tuo programma.
- *Finire in fretta* è spesso la strategia più efficiente.

Grazie per l'attenzione!

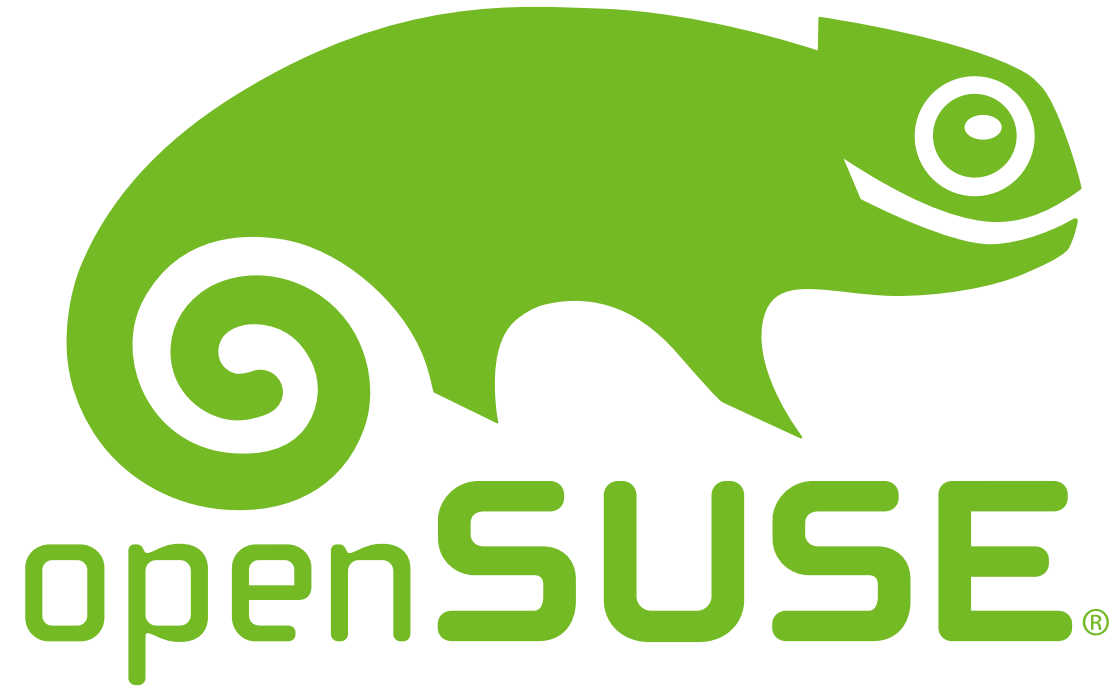
Andrea Manzini

<https://ilmanzo.github.io>

Twitter / GitHub : @ilmanzo

Domande e Risposte

- *"funziona anche su ARM ?"*
- *"posso usarlo su una Virtual Machine o su un container ?"*
- *"E per le GPU ?"*



Risorse

Link e Risorse Utili Documentazione di perf: <https://perf.wiki.kernel.org>

Pagine man: `man powertop` , `man powerstat` , `man perf`

Il wiki di perf: <https://perfwiki.github.io/>

Intel Power Governor (RAPL): [Articolo approfondito](#) sul funzionamento di RAPL.

Brendan Gregg's blog: Una miniera d'oro per tutto ciò che riguarda le performance su Linux. <https://www.brendangregg.com>

`s-tui` : Un'interfaccia testuale per stressare e monitorare la CPU, mostrando frequenza, temperatura e consumo. Utile per osservare il comportamento sotto carico.